



MORPH

MODEL OPTIMIZED REPRESENTATION FOR PROMPT HANDOFFS

A lossless representation compiler for model context

Design, implementation, and offline evaluation
of reversible prompt layout selection

Lossless

Tokenizer measured

Offline first

Evidence gated

Engineering white paper

Specification version 0.1.0

Prepared September 22, 2026

Measured local release evidence

Contents

Abstract	5
Current system state	5
1. Problem and design goal	6
1.1 Why compilation instead of ad hoc formatting	6
1.2 Product boundary	7
2. Preservation model	7
2.1 Tagged intermediate representation	7
2.2 Strict JSON text entry	7
2.3 Safe JavaScript value entry	8
2.4 Equality contract	8
2.5 Semantic identity	8
3. Compiler architecture	8
3.1 Compilation sequence	9
4. Shape profiling and bounded search	10
5. Physical representations	10
5.1 Supported format matrix	10
5.2 Compact JSON baseline	10
5.3 MORPH-framed JSON Lines	11
5.4 Typed delimited rows	11
5.5 Column JSON	11
5.6 Typed path/value	11
5.7 Official TOON adapter	11
6. Artifact, framing, and reconstruction	12
6.1 Machine artifact	12
6.2 Self-contained model bundle	12
6.3 Complete prompt assembly	12
6.4 Integrity versus trust	13
7. Token measurement	13
7.1 Explicit local profile	13
7.2 Counted scope	13

7.3 Budget behavior 14

8. Planning, policy, and evidence14

8.1 Hard gates 14

8.2 Compatibility policy 14

8.3 Economy-experimental policy 14

8.4 Validated policy 14

8.5 Deterministic ranking 15

9. Schema handling15

10. Product surfaces15

10.1 TypeScript SDK 15

10.2 Command-line interface 16

10.3 Browser workbench 16

11. Evaluation design16

11.1 Synthetic fixture families 17

11.2 Property and conformance testing 17

11.3 Provider-neutral quality harness 17

12. Verified offline evidence18

12.1 Test outcomes 18

12.2 Concrete planner behavior 18

12.3 Performance sample 18

12.4 What was not measured 19

13. Security, privacy, and resource controls19

13.1 Untrusted content 19

13.2 Resource defaults 19

13.3 Network and privacy 19

13.4 Schema and artifact safety 20

14. Optional extensions20

14.1 Schema registry 20

14.2 Jev access-pattern classifier 20

14.3 Deferred representation features 20

15. Current limitations and nonclaims20

16. Reproduction21

16.1 Environment and install	21
16.2 Compile and inspect a real fixture	21
16.3 Offline verification	22
16.4 Browser workbench	22
17. Conclusion	22
Repository documentation	22
Standards and primary references	23

Specification version: 0.1.0 Implementation version: 0.1.0 Document date: 2026-09-22 Status: Implemented local release with verified offline evidence Authorship: Project technical document. No individual or automated author is asserted.

Abstract

MORPH, Model Optimized Representation for Prompt Handoffs, is a local representation compiler for structured model context. It accepts JSON-compatible data, an optional JSON Schema, a task description, a target tokenizer profile, and explicit constraints. It then constructs several legal physical representations of the same complete data, independently decodes each applicable representation, reconstructs each finalist from its self-contained model-facing bundle, measures the full rendered prompt with a named tokenizer, applies compatibility, resource, budget, and quality gates, and returns one selected artifact with a deterministic explanation.

The core preservation rule is:

```
semanticEqual(decode(encode(input)), input) == true
```

MORPH applies that rule to the actual self-contained bundle, not to an undisclosed source copy held beside the result. Its native representations are compact JSON, MORPH-framed JSON Lines, typed delimited rows, column-oriented JSON, and typed path/value records. An adapter for the official TOON implementation is also present, but is eligible only when the upstream JavaScript number path can preserve MORPH's stricter number-lexeme contract.

The current release verifies two independent properties offline. First, accepted data can be reconstructed under the documented semantic contract. Second, complete visible prompt text can be counted under the bundled o200k_base tokenizer revision. It does not establish a third property: that a particular language model answers a task as accurately from one representation as from compact JSON. No live model-quality evaluation or qualifying quality profile is bundled, so model comprehension remains untested. This separation is deliberate and visible in the SDK, CLI, workbench, reports, and planner policy.

Current system state

The implementation is operational as a private TypeScript workspace. The release report records the following state:

Capability	Current state
Strict JSON text and JavaScript value intake	Implemented
Tagged, number-preserving IR	Implemented
Five native reversible codecs	Implemented
Official TOON adapter	Implemented with input gating
Complete-context framing and parsing	Implemented
Exact visible-text token counting for the named tokenizer	Implemented
Deterministic planning and explain reports	Implemented
Token and resource budgets	Implemented
TypeScript SDK and CLI	Implemented
Browser comparison workbench	Implemented and local-first
Offline conformance and token benchmark suites	Implemented and executed
Provider-neutral model evaluation runner	Implemented
Live provider model evaluation	Not run

Capability	Current state
Qualified downstream model-quality evidence	None bundled
Optional schema registry	Implemented and tested locally
Optional Jev classifier	Implemented with offline fixtures; live test not run
npm packages or GitHub release	Not published

In practical terms, MORPH can now answer these questions for a concrete request:

- 1 Which registered representations are legal for this input?
- 2 Which applicable representations reconstruct the complete accepted data?
- 3 What exact self-contained text would be handed to a model?
- 4 How many tokens does that complete visible text use under the selected tokenizer revision?
- 5 Which candidates fit the configured limits and prompt budget?
- 6 What quality evidence exists for each nonbaseline family?
- 7 Why did the planner select a representation or remain with compact JSON?

It cannot currently answer whether a target model will understand a smaller layout equally well, because no live matched model evaluation has been run and no quality profile is installed.

1. Problem and design goal

Structured application data is often inserted into a model request as JSON. JSON is widely understood and reconstructable, but its physical layout is not always the smallest possible layout for a particular tokenizer. A uniform table repeats field names. A nested tree may spend text on delimiters and labels at every level. A sparse structure may be more naturally represented as paths. These observations motivate alternative physical layouts, but they do not justify dropping data or assuming that a smaller layout is easier for a model to understand.

MORPH treats representation selection as a compiler problem:

```
application data and optional schema
-> strict JSON-compatible intermediate representation
-> structural profile and task hints
-> bounded legal candidate plans
-> encode, decode, frame, reconstruct, and tokenize
-> compatibility, resource, budget, and quality gates
-> deterministic selection and explain report
-> self-contained model context plus machine artifact
```

The logical value stays stable while the physical text may change. A task hint can affect which layouts are considered or how bounded search is ordered. It never authorizes projection, filtering, sorting, deduplication, aggregation, summarization, translation, rounding, or truncation.

1.1 Why compilation instead of ad hoc formatting

An ad hoc formatter can produce compact-looking text without answering critical questions:

- Can the text reconstruct every row, field, type, and empty container?
- Did a large number retain its exact JSON spelling?
- Are missing properties distinguishable from explicit null values?
- Does the representation include the layout definition and schema meaning it depends on?

- Was the token count measured on the final concatenated prompt or estimated from pieces?
- Is the selected format allowed under the current evidence policy?
- Can another process reproduce the result without the original dataset?

MORPH makes those questions part of the product contract. A candidate is not eligible merely because its payload is short. It must survive the complete pipeline.

1.2 Product boundary

MORPH is a representation compiler. It is not a database engine, retrieval system, query executor, summarizer, agent operating system, model trainer, universal binary codec, or prompt-injection filter. It does not replace native provider tool-call schemas or output schemas.

Re-encoding is not projection. For example, emitting only [country](#) and [revenue](#) from a larger customer record would create a derived dataset, not a lossless physical representation of the original record. A future query or transformation package may deliberately create such a derived dataset with provenance, but that operation is outside the lossless compiler.

2. Preservation model

2.1 Tagged intermediate representation

Every accepted input becomes [morph-ir/1](#), a tagged tree:

```
type IRNode =
  | { kind: "null" }
  | { kind: "boolean"; value: boolean }
  | { kind: "string"; value: string }
  | { kind: "number"; lexeme: string }
  | { kind: "array"; items: readonly IRNode[] }
  | {
    kind: "object";
    entries: readonly (readonly [string, IRNode])[];
  };

interface MorphIR {
  irVersion: "morph-ir/1";
  root: IRNode;
  inputKind: "json-text" | "js-value";
  semanticDigest: string;
}
```

Objects use entry arrays rather than unsafe ordinary property assignment. This allows keys such as [__proto__](#), [constructor](#), [prototype](#), the empty string, numeric-looking text, slashes, tildes, and delimiter characters to remain ordinary data.

2.2 Strict JSON text entry

[compileJson](#) routes text through a dedicated parser before any native number conversion. The parser detects duplicate object keys before a map can overwrite one, validates JSON number grammar, retains the exact numeric token, enforces byte, depth, and node limits while parsing, rejects malformed escapes and trailing content, and rejects lone Unicode surrogates.

For JSON text, the following are distinct preserved numbers:

```
1
1.0
1e0
1E+0
```

```
-0
1.2300
9007199254740993
```

Source whitespace, object member display order, and equivalent string escape spellings are not preserved. This is semantic preservation under a documented contract, not byte-for-byte source preservation.

2.3 Safe JavaScript value entry

`compileValue` accepts only a documented JSON-compatible subset. It reads property descriptors rather than invoking getters, and it never calls `toJSON`. It rejects cycles, accessors, non-enumerable object fields, class instances, unsupported prototypes, sparse arrays, custom array properties, functions, symbols, `undefined`, `BigInt`, and nonfinite numbers.

A JavaScript number may already have lost precision before MORPH receives it. The adapter cannot reconstruct information that is no longer present. Exact large JSON numbers should therefore enter through JSON text. Negative zero is handled explicitly.

Arbitrary hostile proxies are outside the JavaScript-value boundary. Untrusted input should use the JSON-text path.

2.4 Equality contract

`semanticEqual` requires:

- identical object key sets and recursively equivalent values;
- identical array length, item order, and duplicate multiplicity;
- identical string contents, including whitespace and line endings;
- identical boolean and null values and types;
- identical number lexemes;
- separate treatment of absent properties, explicit null, false, empty strings, and empty containers.

Object display order is not semantically significant. Array order is always significant.

MORPH does not normalize Unicode. A precomposed accented string and a decomposed combining-mark sequence remain distinct. The parser accepts well-formed Unicode scalar strings and rejects lone surrogates rather than silently rewriting them.

2.5 Semantic identity

The semantic digest is SHA-256 over the versioned `morph-c14n/1` canonical representation. Canonicalization includes explicit type tags, unambiguous UTF-8 lengths, array order, numeric lexemes, and locale-independent object-key sorting.

The digest detects mismatches. It is not a signature, an authentication mechanism, or proof that the content is safe.

3. Compiler architecture

The dependency direction is intentionally narrow:

```
+-----+
| CLI and browser workbench |
+-----+
      |
      v
+-----+
| composed local SDK        |
+-----+
      |
```




Core owns request validation, the IR, semantic equality, profiling, planning, artifact construction, framing, rendering, verification, error types, and orchestration. Encoders and tokenizers implement explicit interfaces. The CLI supplies filesystem behavior. The workbench supplies browser UI and worker isolation. The evaluation package calls the compiler, while the compiler never launches a live evaluation.

No artifact can cause executable plugins to load. Encoder and tokenizer registration happens in application code.

3.1 Compilation sequence

For every compile request, MORPH performs these stages:

- 1 Runtime-validate the request and merge bounded defaults.
- 2 Resolve the target profile and tokenizer by exact registered identity.
- 3 Parse JSON text or adapt a supported JavaScript value into [morph-ir/1](#).
- 4 Sanitize and locally validate an optional Draft 2020-12 schema.
- 5 Profile the complete accepted IR.
- 6 Enumerate a stable bounded candidate list, with compact JSON first.
- 7 Check each candidate's applicability against the complete input.
- 8 Encode the complete IR and enforce payload limits.
- 9 Decode the section and compare it semantically with the source IR.
- 10 Create a self-contained artifact containing required interpretation dependencies.
- 11 Render the canonical self-contained bundle and complete prompt.
- 12 Parse the bundle without the original source, decode it again, and compare its digest and semantics.
- 13 Count the entire rendered prompt in one tokenizer operation.
- 14 Apply allowlist, forced-format, experimental-feature, quality, and token-budget gates.
- 15 Select deterministically or return a typed failure.

An unexpected failure from an encoder that declared itself applicable is a bug signal. The default behavior is fatal. An explicit quarantine mode can record that candidate as rejected and continue, but it cannot excuse a failed compact JSON baseline or bypass any semantic, budget, policy, or integrity check.

4. Shape profiling and bounded search

The profiler records structural facts, not samples intended for public telemetry. It reports root kind, node count, maximum depth, object and array counts, scalar types, string-length statistics, boolean and null counts, repeated-scalar statistics, and record-array observations. For record arrays it records record counts, field sets, observed field types, missing-key counts, and explicit-null counts.

Missingness and null are kept separate. A property absent from most records is not treated as a property present with `null`.

Structural counts are exact for an accepted bounded input. Scalar-cardinality tracking is capped at 4,096 entries by default. When that cap prevents tracking a new scalar, the profile reports `exact: false`, sets a cap flag, and increments an untracked count rather than claiming complete cardinality information.

Candidate enumeration is deterministic and bounded. Default limits include at most 24 candidate plans and at most eight retained materialized candidate artifacts. The built-in native order is compact JSON, JSON Lines, four typed-row delimiters, columns JSON, and path/value. The composed SDK also registers three gated TOON delimiter plans. Plans beyond the cap are reported as pruned, so MORPH never presents a bounded search as a global optimum across unsearched formats.

Estimates may prune obvious nonstarters, but any finalist used for selection is fully encoded, independently decoded, framed, reconstructed from the bundle, and tokenized.

5. Physical representations

5.1 Supported format matrix

Representation	Version	Applicability	Important preservation behavior
Compact JSON	<code>json-compact@1</code>	Every accepted IR	Prints strict JSON directly from tagged nodes and retains number lexemes
JSON Lines	<code>json-lines@1</code>	Root arrays, including empty arrays	Declares root framing and element count; one compact JSON value per physical line
Typed delimited rows	<code>rows-delimited@1</code>	Nonempty uniform arrays of primitive-valued objects	Declares fields, row count, delimiter, and JSON-scalar cell codec
Column JSON	<code>columns-json@1</code>	Same uniform primitive record arrays	Stores one aligned JSON array per field while retaining source row order
Typed path/value	<code>path-value@1</code>	Every accepted IR	Emits a typed record for every node, including empty containers
Official TOON adapter	<code>toon@4.1.1</code>	Only inputs proven to preserve MORPH equality	Uses the official package with strict decode and numeric gating

5.2 Compact JSON baseline

Compact JSON supports every accepted IR and is the primary baseline for savings. It removes unnecessary structural whitespace but does not convert number nodes through JavaScript numbers. Its interpretation guide is intentionally minimal, so alternative formats do not receive an unfair comparison against artificially verbose JSON.

5.3 MORPH-framed JSON Lines

JSON Lines supports a root array. The metadata declares the root type, element count, and framing version. Every element is one complete compact JSON value per physical line. Newline characters inside strings remain escaped and therefore do not create extra records. The zero-length array has an explicit count and an empty payload.

The decoder rejects carriage returns, trailing line feeds, missing or extra lines, empty nonzero lines, count mismatches, and resource-limit violations. This is a MORPH-framed representation, not a claim that an unframed JSON Lines stream preserves its root type by itself.

5.4 Typed delimited rows

Rows apply only to a nonempty root array of objects with identical key sets, at least one field, and primitive cells. Four bounded delimiter plans are available: tab, comma, pipe, and semicolon.

Field names appear once in metadata. Each cell is one JSON scalar token. Strings are always quoted and escaped as JSON strings. Numbers keep their lexemes. Booleans and null use literal JSON spelling. As a result, `123` remains distinct from `"123"`, `null` remains distinct from `"null"`, and an empty string remains a present string value.

The decoder scans character by character and recognizes separators only outside quoted strings. It does not call a naive string split. It rejects invalid cells, unterminated strings, row-width errors, duplicate fields, row-count mismatches, unsafe declared sizes, and unexpected rows.

5.5 Column JSON

Columns apply to the same uniform primitive record arrays. The payload is a compact JSON array containing one array per declared field. Index `i` across all columns reconstructs source row `i`.

The decoder requires unique field names, the exact declared column count, a common row length, primitive cells, and bounded allocations. MORPH does not assume that columns improve aggregation or comparison tasks. The layout can be smaller for a tokenizer while increasing the model's alignment burden. That tradeoff belongs to model-quality evaluation.

5.6 Typed path/value

Path/value supports every accepted IR. It uses RFC 6901 JSON Pointer paths and emits a typed JSON Lines record for every node. Container nodes are explicit, including empty objects and arrays. Number nodes carry a numeric type tag and the original lexeme as a string.

The empty string identifies the root. Pointer segments encode `~` as `~0` and `/` as `~1`. Parent container records distinguish an object key named `"0"` from array index zero.

The decoder rejects duplicate paths, invalid pointer escapes, noncanonical pointers, missing parents, children under primitive nodes, conflicting container declarations, incorrect record arity, invalid number tokens, holes, out-of-range indices, multiple roots, and declared sizes beyond configured limits.

5.7 Official TOON adapter

MORPH uses `@toon-format/toon` 4.1.1 rather than inventing a TOON-like syntax. It evaluates comma, tab, and pipe delimiter options with indent size 2 and strict decoding.

The upstream host interface uses JavaScript numbers. MORPH therefore rejects a TOON plan when a number is nonfinite after conversion, is negative zero, changes when converted through `Number`, or is an unsafe integer. Lexemes such as `-0`, `1.2300`, `1e3`, and precision-unsafe integers are inapplicable unless the exact round trip is demonstrably preserved. Native representations remain available.

The adapter performs an actual official encode and strict decode during applicability checks, applies MORPH semantic equality, and requires canonical re-encoding on decode. MORPH does not adopt upstream benchmark claims as MORPH results.

6. Artifact, framing, and reconstruction

6.1 Machine artifact

The selected result is [morph-artifact/1](#). It records:

- the input semantic digest and input kind;
- the selected encoding, format version, and physical options;
- the encoded payload and layout metadata;
- the versioned interpretation guide and optional schema dependency;
- a self-contained dependency mode;
- the exact target profile;
- the task, prefix, suffix, and prompt-template version;
- payload, dependency, and artifact identity digests.

The artifact does not hide a source JSON copy solely to make decoding pass. [decode](#) resolves the registered decoder, validates integrity, reconstructs the IR from the encoded section, and checks the input semantic digest. [decodeJson](#) prints valid compact JSON directly from the reconstructed IR.

6.2 Self-contained model bundle

[MORPH-CONTEXT/1](#) contains the compiler-owned interpretation guide, canonical layout metadata, optional schema, and encoded payload. It uses deterministic collision-checked section markers. A boundary is accepted only when its complete marker strings do not occur inside any framed section.

[parseModelContext](#) validates the header and boundary grammar, requires every section exactly once, checks the unique final marker, limits bundle allocation, parses metadata and schema as strict JSON, and passes the resulting section to the registered codec decoder.

The framing makes parsing unambiguous. It does not neutralize prompt injection and does not make schema descriptions or dataset values trusted instructions.

6.3 Complete prompt assembly

[MORPH-PROMPT/1](#) assembles, in order:

```
caller prefix
complete self-contained bundle
task instruction
caller suffix
```

Each part is framed with deterministic collision-checked markers. The selected tokenizer receives the complete assembled [rendered](#) string in one call.

The model context API also exposes the guide, data block, self-contained bundle, rendered text, and rendered SHA-256 digest. Machine-only explain details and performance statistics remain outside the prompt unless an application explicitly sends them.

6.4 Integrity versus trust

Artifact verification recomputes the payload digest, dependency digest, and artifact identity. It checks the plan, section, guide, and target bindings, then reparses and decodes the self-contained bundle.

The result distinguishes checksum validity, dependency completeness, and semantic round trip. A matching unkeyed hash detects consistency but does not authenticate who created the artifact or prove that its instructions are safe.

7. Token measurement

7.1 Explicit local profile

The bundled profile is:

```
profile: local-o200k-base
tokenizer: o200k_base
implementation: js-tiktoken 1.0.21
revision: js-tiktoken@1.0.21:o200k_base:sha256:446a9538cb6c348e3516120d7c08b09f57c36495e2acfffe59a5bf8b0cfb1a2d
normalization: none
special-token-looking input: ordinary text
runtime asset download: none
model binding: none
provider binding: none
```

The JavaScript implementation is accurately labeled as a third-party port. Its bundled ranks and expanded vocabulary digests are checked locally. Seven committed fixture cases were also checked against official Python [tiktoken](#) 0.14.0 [encode_ordinary](#), with seven matches and zero mismatches. Python is a development oracle, not a runtime dependency.

7.2 Counted scope

Every finalist is counted as one complete visible rendered prompt. The count includes:

- caller prefix and suffix;
- compiler interpretation guide;
- format declaration and field names;
- included schema and descriptions;
- self-contained framing;
- payload;
- task instruction.

The final count is not obtained by adding separately tokenized components because tokenization can merge across component boundaries. The adapter reports:

```
scope: rendered-text
certainty: exact-for-tokenizer
```

This establishes an exact count for that visible string under that tokenizer revision. It does not establish exact billed or context-window tokens for a provider request with hidden instructions, role framing, tools, images, output schemas, or provider-specific accounting.

Unknown model identifiers never silently map to the local tokenizer. Character count divided by four is never presented as an exact tokenizer result.

7.3 Budget behavior

`maxPromptTokens` applies to the declared visible rendered-text scope. A candidate over the budget is ineligible. If no eligible candidate fits, compilation returns `BUDGET_EXCEEDED` with the smallest verified count where available. It never silently truncates the data and never returns an over-budget compact JSON artifact as success.

The local target has no context-window or provider-overhead binding. A reserved output token setting that requires unavailable context accounting is therefore rejected instead of treating unknown overhead as zero.

8. Planning, policy, and evidence

8.1 Hard gates

Before cost ranking, a candidate must pass:

```
input accepted
-> complete-input applicability
-> resource limits
-> direct decode and round trip
-> self-contained dependency completeness
-> bundle parse, decode, and round trip
-> exact tokenizer and target binding
-> experimental-feature policy
-> applicable quality policy
-> actual rendered prompt budget
-> final ranking eligibility
```

Every rejection retains reason codes. Typical codes include `ENCODER_NOT_APPLICABLE`, `ROUNDRIP_FAILED`, `SELF_CONTAINED_ROUNDRIP_FAILED`, `QUALITY_UNKNOWN`, `QUALITY_PROFILE_MISMATCH`, `OVER_TOKEN_BUDGET`, `BELOW_MINIMUM_SAVINGS`, `ENCODING_NOT_ALLOWED`, and `CANDIDATE_LIMIT_PRUNED`.

8.2 Compatibility policy

`compatibility` is the default. Without applicable quality evidence, compact JSON is the only automatically selectable family. Smaller alternatives still appear in `compare()` with real counts and reasons, but unknown model quality prevents their automatic deployment.

This is a conservative authority decision, not a claim that compact JSON is always best.

8.3 Economy-experimental policy

`economy-experimental` may select among reversible, supported alternatives using measured complete-prompt tokens. It explicitly labels model comprehension as unverified when no quality evidence exists.

By default, an unforced nonbaseline candidate must save both at least 16 tokens and at least 2 percent relative to compact JSON. A caller can configure these thresholds. The hysteresis prevents changing formats for negligible differences and guide overhead noise.

8.4 Validated policy

`validated` requires an applicable quality profile before selecting a nonbaseline family. A profile must match the target model and tokenizer identity, encoding and options, guide and renderer versions, task family, workload scope, evidence digest, case count, independent dataset count, and validity period.

No quality profile ships with the current release. Therefore validated mode remains with compact JSON when it fits. It does not weaken policy silently.

8.5 Deterministic ranking

Eligible plans are ranked by the objective and stable tie breakers. For the implemented prompt-token objective, ordering is:

- 1 Lower complete rendered token count.
- 2 Compact JSON preference.
- 3 Fewer interpretation mechanisms.
- 4 Lexically stable plan ID.

An explicitly forced encoding bypasses cost ranking and minimum-savings hysteresis. It does not bypass semantic correctness, applicability, policy, tokenizer, resource, quality, or budget gates.

The [estimated-request-cost](#) objective is not currently available because no versioned provider pricing and full request-accounting profile is registered. MORPH returns [PRICING_PROFILE_REQUIRED](#) rather than silently substituting a token objective.

9. Schema handling

A schema is optional. The current implementation accepts an object or Boolean JSON Schema, supports Draft 2020-12, rejects nonlocal [\\$ref](#) references, and performs no network resolution. Ajv 8.20 runs in strict mode with all-errors reporting and without type coercion, default insertion, property removal, format side effects, or data mutation.

When supplied, the complete sanitized schema is included in every candidate's self-contained bundle and counted in the final prompt. Structural column labels do not replace caller-provided descriptions or other schema meaning.

JSON Schema numbers are mathematically broader than JavaScript binary64 numbers. When exact validation would depend on a precision-sensitive number and a numeric comparison, equality, [multipleOf](#), or [uniqueItems](#) rule, MORPH returns [SCHEMA_NUMERIC_VALIDATION_UNSUPPORTED](#). It does not round the value and report exact validation.

Schema descriptions and property names remain untrusted data even when they resemble instructions.

10. Product surfaces

10.1 TypeScript SDK

The composed local entry point is:

```
import { createDefaultMorph } from "@morph/sdk";

const morph = createDefaultMorph();
```

It registers the five native codecs, gated official TOON adapter, local tokenizer, local target profile, and an empty quality-profile list. Advanced applications can call [createMorph](#) and register an explicit custom set.

The main methods are:

```
morph.compile(request)
morph.compileJson(text, options)
morph.compileValue(value, options)
morph.compare(request)
morph.decode(artifact)
morph.decodeJson(artifact)
morph.renderModelContext(artifact)
morph.verify(artifact)
```

`compile` returns either a selected artifact and explain report or a typed `MorphError`. `compare` returns candidate facts, including inapplicable and ineligible alternatives, without returning a selected artifact. Lower-level parsing, profiling, framing, canonicalization, schema, source-map, and validation APIs are also exported.

10.2 Command-line interface

The workspace CLI exposes:

```
inspect
compare
compile
render
decode
verify
bench --suite conformance|tokens|performance --offline
doctor
```

It accepts standard input with `--input -`, writes machine results to standard output and diagnostics to standard error, refuses overwrite unless `--force` is explicit, performs atomic local writes, checks that output paths remain inside the selected workspace, bounds reads, rejects malformed UTF-8, and never uploads input or output.

`doctor` verifies the local Node range, registered encoders, tokenizer smoke behavior, and tokenizer asset digests without reading credentials or making a billable call. Missing optional Jev access is informational.

The CLI uses distinct nonzero exit classes for invalid configuration, registration or tokenizer failures, artifact and decode failures, budgets and resource limits, and optional integration failures. No bundled command launches a model evaluation or Jev request.

10.3 Browser workbench

The React/Vite workbench invokes the real compiler in a module worker. It provides synthetic examples, JSON and optional schema input, task and access-pattern controls, target and policy selection, prompt budgets, a candidate table, selected context, machine artifact, decoded JSON, and a preservation view.

The workbench clearly separates:

```
Data preservation: verified, failed, or not run
Token measurement: exact text, provider reported, or estimated
Model comprehension: untested, evidence-qualified, or failed
Selection policy: compatibility, experimental, or validated
```

Each request has a monotonically increasing ID and SHA-256 content digest. Editing input, importing another artifact, or cancelling clears prior results and terminates current worker work, so a stale response cannot be shown for new input.

The workbench bundles its tokenizer assets, makes no compiler network call, has no account system, analytics, provider integration, database, or remote font dependency, and creates downloads only after a user action. Imported artifacts are parsed, verified, rendered, and decoded locally before display.

11. Evaluation design

MORPH treats three evidence layers as independent:

Layer	Question	Current status
A. Codec conformance	Can software reconstruct the accepted data exactly?	Implemented and executed offline
B. Prompt efficiency	How many tokens does the complete rendered prompt use under the named tokenizer?	Implemented and executed offline

Layer	Question	Current status
C. Model task quality	Does a specific model answer matched tasks correctly from each representation?	Harness implemented; live evaluation not run

A pass at Layer A does not imply model understanding. A lower count at Layer B does not imply accuracy or lower provider billing. Layer C requires actual matched model executions or clearly identified imported results.

11.1 Synthetic fixture families

The versioned `synthetic-v2` corpus and committed fixtures cover:

- narrow and wide uniform tables;
- nested objects and arrays with empty containers;
- sparse records with absent, null, false, and empty-string states;
- repeated categories with different cardinalities;
- identifier-heavy data and leading-zero strings;
- large integers, exponents, negative zero, and exact decimal lexemes;
- emoji, Chinese text, accents, combining marks, delimiters, and newlines;
- ordered duplicate events and position-sensitive sequences;
- tiny payloads where guide overhead dominates;
- prompt-like strings, marker-shaped content, and hostile-looking keys.

Ground-truth generators cover single-record lookup, cross-record comparison, multi-field filtering, nested-path lookup, missing-versus-null questions, order-sensitive questions, and small exact aggregations.

11.2 Property and conformance testing

The extended property suite uses seed `20260921`. Its recorded 5,000-property budget generated 2,500 general inputs and 625 uniform tables, exercising at least 5,000 complete native bundle round trips. Property tests complement explicit malformed, numeric, Unicode, prototype-key, and framing fixtures. A finite number of generated cases is evidence from sampled executions, not a formal proof.

Offline conformance enumerates every native and gated TOON plan for each manifest fixture. Applicable plans must directly round trip. A full selected artifact is also rendered, parsed from its self-contained bundle, decoded, and compared with the original semantics. A declared inapplicability is recorded separately from a failure.

11.3 Provider-neutral quality harness

The evaluation package can prepare matched compact-JSON and candidate contexts, randomize trial order from a seed, call an application-supplied evaluator, and enforce request, concurrency, retry, timeout, output-token, and call-or-priced-cost bounds. It keeps failures, refusals, truncations, invalid output, and retries in complete denominators.

No provider client or credential is bundled. Calling the harness without an evaluator produces a `not-run` manifest and performs no network request.

For binary correctness, the paired helper analyzes:

```
delta = accuracy(candidate) - accuracy(compact JSON)
```

It implements a seeded paired dataset-cluster bootstrap. It averages matched differences within each dataset and resamples dataset means, so repeated calls on one dataset are not treated as independent datasets. Qualification requires prespecified case and dataset minima and a lower confidence bound greater than the negative allowed-regression margin.

The helper analyzes caller-supplied records. It does not prove that the dataset IDs are honest, validate preregistration, run a model, or install a quality profile by itself.

12. Verified offline evidence

All figures in this section are recorded execution results from [RELEASE_REPORT.md](#) and curated synthetic reports. They are scoped evidence, not universal product claims.

12.1 Test outcomes

Verification activity	Recorded outcome
Ordinary test suite	24 files, 130 tests passed, 0 failed, 0 skipped
Extended seeded suite	5 files, 28 test definitions passed, seed 20260921
Complete native bundle property round trips	At least 5,000
Offline conformance	73 cases total, 70 passed, 0 failed, 3 correctly inapplicable
Complete-prompt token benchmark	61 measured candidate rows across 9 fixtures, 58 policy-eligible
Official Python tokenizer oracle	7 of 7 fixture cases matched, 0 mismatches
Dependency audit	0 advisories across 222 installed dependencies at the recorded run

The three inapplicable conformance cases were not hidden failures. They were TOON plans gated from a precision-sensitive input that could not satisfy MORPH's number-lexeme contract.

12.2 Concrete planner behavior

For the recorded customer comparison request, complete visible prompt counts under the named local tokenizer were:

Candidate	Tokens	Difference from compact JSON
Compact JSON	896	baseline
JSON Lines	884	12 fewer, 1.339 percent
Column JSON	891	5 fewer, 0.558 percent

Under the default experimental hysteresis, compact JSON remained selected because JSON Lines did not meet either the 16-token threshold or the 2 percent threshold. This result demonstrates that MORPH can reject a smaller candidate for an explicit policy reason. It does not demonstrate that JSON is universally better.

For the recorded sequence fixture, column JSON measured 704 tokens and compact JSON measured 736 tokens. The difference was 32 tokens, or 4.348 percent, for that exact complete prompt and tokenizer revision. This is Layer B token evidence only. No model was asked to answer the sequence task from either representation.

12.3 Performance sample

The recorded local environment was Node.js 24.19.0 on Windows x64 with an AMD Ryzen 7 5800XT and 16 logical CPUs. The benchmark used three warmups and 20 samples per stage across 15 stages, for 300 timing samples.

Measurement	Median	p95
Full comparison	16.2204 ms	17.1545 ms
Warm tokenization	0.8470 ms	0.9286 ms

Cold tokenizer load was 345.1169 ms. The recorded approximate heap delta was 172,030,336 bytes. That heap value is a before-and-after approximation, not peak memory. These results describe one documented machine and fixture, not a general latency promise.

12.4 What was not measured

The release ran no paid or free provider model evaluation. It produced no downstream accuracy, noninferiority certification, provider latency, provider usage, provider cost, prompt-cache result, or retry-rate result. It bundled no quality profile. The correct current label is:

```
model task quality: not run
model comprehension: unbound
```

13. Security, privacy, and resource controls

13.1 Untrusted content

All dataset values, keys, schemas, imported artifacts, registry bundles, and optional remote responses are untrusted data. MORPH never evaluates input, constructs executable functions from it, interpolates it into shell commands, interprets JSON Pointers as filesystem paths, or promotes it into privileged prompt roles.

Encoding does not remove prompt injection. An instruction-shaped string remains instruction-shaped after it becomes a row cell, column value, TOON value, or path record. Applications remain responsible for provider roles, least-privilege tools, output validation, authorization, and incident response.

13.2 Resource defaults

Core defaults are:

```
maximum input bytes: 5 MiB
maximum depth: 64
maximum nodes: 250,000
maximum candidates: 24
maximum rendered bytes per candidate: 16 MiB
planning abort limit: 5,000 ms
network permission: false
```

Decoders also bound payload bytes, node counts, declared array and table sizes, path records, and allocations. Core cancellation and the planning deadline use cooperative checkpoints between synchronous stages. The browser provides a harder local stop by terminating its worker.

13.3 Network and privacy

Core compilation, native codecs, tokenizer operation, the TOON adapter, CLI local commands, offline suites, and workbench compilation make no application-level network calls. Tokenizer ranks are bundled. There is no mandatory telemetry or automatic input upload.

Setting `allowNetwork: true` creates no capability by itself. Network-capable code is isolated in optional adapters and requires additional explicit configuration.

The CLI bounds and validates file reads, refuses unexpected overwrite, writes atomically, and validates output paths against the selected workspace. These controls are local safeguards, not a multi-tenant authorization boundary.

13.4 Schema and artifact safety

Remote schema resolution is disabled. Unknown envelope versions and unregistered encodings fail closed. Native decoders reject unknown metadata, malformed counts, duplicate declarations, invalid scalar grammar, and unsafe allocation requests rather than guessing or repairing.

Artifacts use consistency hashes but no signature. Applications that require producer authentication need a separate signing and key-management layer.

14. Optional extensions

14.1 Schema registry

The optional registry stores content-addressed schema and guide bundles in memory or in a caller-selected local directory. It enforces entry and total-size bounds, uses digest-derived identities, verifies content on resolution, and supports hydration of a reference artifact back into a self-contained artifact.

A registry reference is not directly model-ready. Hydration must resolve the exact bundle, verify its digest, rebuild self-contained dependencies, parse the normal artifact, and pass integrity verification before rendering. A registry hit does not mean a model retained the schema and does not remove schema tokens from default prompt counting.

14.2 Jev access-pattern classifier

The optional Jev package implements a bounded task classifier for these labels:

```
entity_lookup
multi_entity_comparison
aggregation
filtering
nested_path_lookup
sequence_analysis
unknown
```

It requires a local/server credential, explicit network permission, and explicit permission to disclose the task text. Raw dataset values are excluded by default. It validates the exact option set, finite probabilities, probability sum, winning choice, confidence, resolved model string, and token usage, and returns a deterministic `unknown` fallback on disabled access or failure.

The adapter's live contract test is not run. No service availability or resolved production model is claimed. Jev confidence measures concentration in its returned distribution. It is not downstream answer accuracy, cannot enter `QualityEvidence` as an accuracy number, and cannot make an ineligible representation eligible.

14.3 Deferred representation features

Dictionary columns, explicit missingness masks, hybrid layouts, retained-provider-context mode, provider prompt-cache accounting, binary formats, packed bitmaps, delta encoding, query execution, and learned cost models are not implemented in the native release. They are not listed as supported features.

15. Current limitations and nonclaims

- 1 Model quality is untested. Smaller prompts may be harder for a model to interpret.
- 2 The bundled tokenizer counts the complete visible prompt only. It does not count provider-hidden framing, tools, images, or billing-specific overhead.
- 3 The local profile has no verified model or provider binding.
- 4 No provider adapter, provider pricing profile, or production request-cost objective is bundled.

- 5 Quality-profile gates exist, but no profile is bundled and core does not authenticate external evidence provenance.
- 6 Exact arbitrary-precision schema numeric validation is conservatively rejected for unsupported keyword combinations.
- 7 TOON is inapplicable when its JavaScript number path cannot preserve MORPH number lexemes.
- 8 Source maps currently cover compact JSON only and use UTF-16 code-unit offsets.
- 9 Core cancellation cannot interrupt one synchronous codec or tokenizer operation already in progress.
- 10 The CLI is a local tool and not a multi-tenant service boundary.
- 11 Schema registry references require verified hydration before model use.
- 12 The optional Jev adapter has only offline synthetic contract evidence.
- 13 No universal token reduction, cost reduction, accuracy improvement, cache discount, or latency guarantee is claimed.
- 14 No npm package, public repository, or GitHub release is implied by the local workspace.

16. Reproduction

16.1 Environment and install

The repository requires Node.js `>=22.12.0 <25` and pnpm 10.15.0.

```
pnpm install --frozen-lockfile
pnpm morph doctor
```

16.2 Compile and inspect a real fixture

```
pnpm morph inspect --input fixtures/examples/customers.json

pnpm morph compare \
  --input fixtures/examples/customers.json \
  --task-file fixtures/examples/comparison.task.txt \
  --schema fixtures/examples/customers.schema.json \
  --target local-o200k-base \
  --policy economy-experimental \
  --report reports/comparison.json

pnpm morph compile \
  --input fixtures/examples/customers.json \
  --task-file fixtures/examples/lookup.task.txt \
  --target local-o200k-base \
  --policy compatibility \
  --output artifacts/customers.morph.json \
  --report reports/customers-explain.json

pnpm morph render \
  --input artifacts/customers.morph.json \
  --output artifacts/customers.context.txt

pnpm morph verify --input artifacts/customers.morph.json

pnpm morph decode \
  --input artifacts/customers.morph.json \
  --output artifacts/customers.restored.json
```

Existing output paths are not overwritten unless `--force` is supplied.

16.3 Offline verification

```
pnpm lint
pnpm typecheck
pnpm test
pnpm test:extended
pnpm build
pnpm smoke:package
pnpm bench:conformance
pnpm bench:tokens
pnpm bench:performance
```

The benchmark commands are offline. They do not call a model. Current recorded results are in [RELEASE_REPORT.md](#) and curated files under [reports/examples/](#).

16.4 Browser workbench

```
pnpm workbench
```

The workbench uses bundled assets and the real compiler. A normal local session does not need an API key, account, provider, registry service, or network access after dependencies are installed.

17. Conclusion

MORPH's current result is not a universal replacement for JSON. It is a working, evidence-aware compiler that can prove software reconstruction for supported formats, show the exact self-contained text a model would receive, count that text under an identified tokenizer, enforce budgets and policy, and explain every selection or rejection.

The central engineering value is disciplined separation:

- preservation is established by strict parsing, decoding, semantic comparison, and reconstruction from the rendered bundle;
- prompt size is established by complete-text measurement under a versioned tokenizer;
- downstream model quality remains unknown until matched model evaluations produce applicable evidence.

That separation makes compact JSON a valid winner, makes a smaller but unqualified candidate visible without silently deploying it, and provides a reproducible path for future model-specific qualification.

Repository documentation

- [Architecture](#)
- [Semantic contract](#)
- [Artifact, framing, and codec formats](#)
- [TypeScript SDK API](#)
- [Command-line interface](#)
- [Benchmarking and evidence](#)
- [Security and privacy](#)
- [Dependencies and external references](#)
- [Release process](#)
- [Verified release report](#)
- [Canonical implementation specification](#)

Standards and primary references

The repository's reviewed source notes are in [REFERENCES.md](#). The main external references are:

- 1 RFC 8259, The JavaScript Object Notation Data Interchange Format: <<https://www.rfc-editor.org/rfc/rfc8259.html>>
- 2 RFC 6901, JavaScript Object Notation Pointer: <<https://www.rfc-editor.org/rfc/rfc6901.html>>
- 3 TOON API, specification, and official implementation: <<https://toonformat.dev/reference/api>>, <<https://github.com/toon-format/spec>>, and <<https://github.com/toon-format/toon>>
- 4 OpenAI tiktoken and token-counting scope: <<https://github.com/openai/tiktoken>> and <<https://developers.openai.com/api/docs/guides/token-counting>>
- 5 JSON Schema Draft 2020-12: <<https://json-schema.org/draft/2020-12>>
- 6 fast-check property-testing documentation: <<https://fast-check.dev/>>
- 7 TypeSafe Jev primitives, API, and confidence semantics: <<https://docs.typesafe.ai/primitives>>, <<https://docs.typesafe.ai/api>>, and <<https://docs.typesafe.ai/confidence>>
- 8 Provider prompt-caching example, used only as a provider-specific reference: <<https://developers.openai.com/api/docs/guides/prompt-caching>>

These references support format, tokenizer, validation, and optional-integration choices. They do not establish MORPH token savings, model accuracy, provider availability, or a successful live integration.